ME46060 Engineering Optimization

Exercise 3 & 4

Course Code	ME46060	
Course Name	Engineering Optimization	
Student Names	Marcos Carrera Valle	5682525
	Nicolás Nogueiras García	5683408
Responsible Instructor	Prof.dr.ir. M. Langelaar	
	Dr.ir. L.F.P. Noel	
	Dr. O. Nejadseyfi	
Report Date	March 8, 2026	

Exercise 3

3.1 Line searches using Figure 1

In order to calculate the optimal step size (α), the relationship between the starting point and the optimal point is defined by $x = x_q + \alpha S_q$. It can be solved for α using simple algebra for whichever coordinate changed the most:

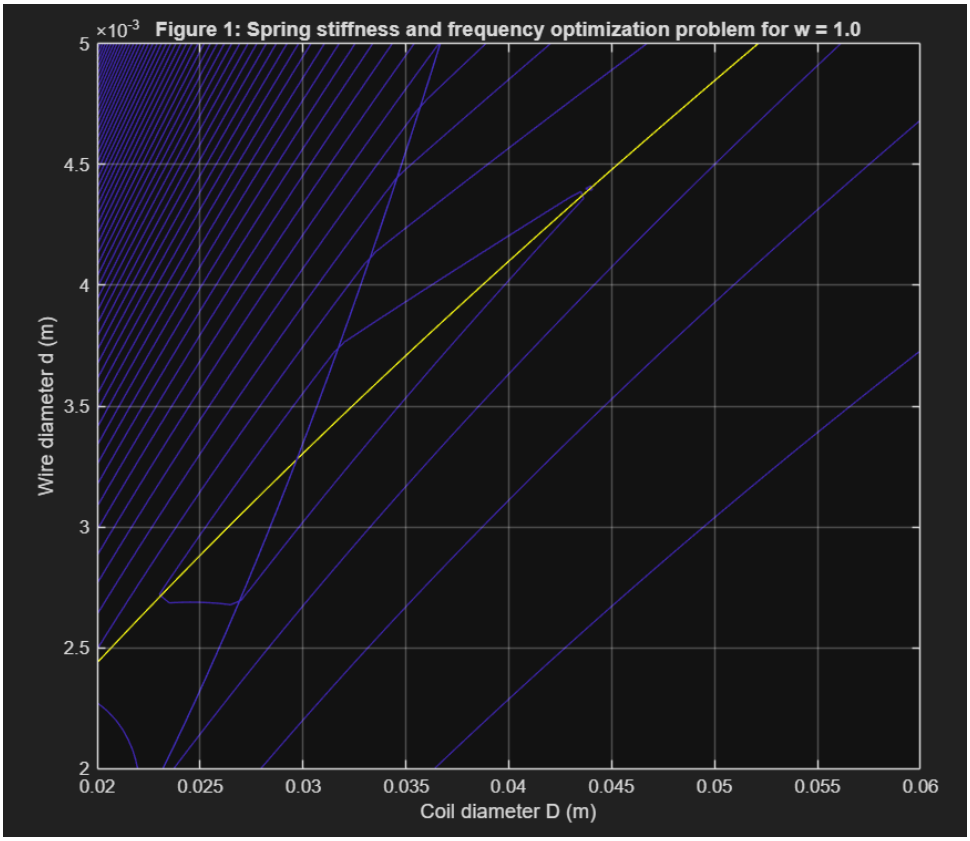

Figure 1: Spring stiffness and frequency optimization problem for w = 1.0

For S_{q1} (Horizontal):

$$\alpha = \frac{D_{estimated} - 0.022}{0.002}$$

For S_{q2} (Vertical):

$$\alpha = \frac{d_{estimated} - 0.004}{-0.0005}$$

For S_{q3} (Diagonal): The D or d formula above can be used since they should yield the same α if the point is exactly on the line.

This yields the following table:

Table 1: Visual estimation results for line search directions ($x_q = [0.022, 0.004]$)

Direction	**Est.** D **(m)**	**Est.** d **(m)**	**Obj.** $f(\alpha)$	**Est.** α
S_{q1} (Horiz.)	0.04	0.00400	0.422	9.00
S_{q2} (Vert.)	0.0220	0.00276	0.76707	2.50
S_{q3} (Diag.)	0.0260	0.00308	0.38426	2.00

3.2 Line searches using the line search algorithm fminbnd

This exercise investigated the application of one-dimensional line searches to the optimization of a valve spring design. The objective function $f(x)$ was minimized along three specific search directions starting from $x_q = [0.022, 0.004]$. A comparison between initial visual estimations from the contour plot (Figure 1) and the numerical results obtained via the `fminbnd` algorithm revealed that diagonal searches (S_{q3}) provide the most efficient path toward the global minimum, yielding a minimized objective value of approximately 0.1754. Furthermore, the accuracy of the estimation was not perfect and could be improved with an increase in the density of the contour lines; however, it was good enough.

Table 2: Comparison of Visual Estimates (Ex 3.1) and fminbnd Results (Ex 3.2)

Direction	**Est.** α	**Opt.** α	**Est.** $f(\alpha)$	**Opt.** $f(\alpha)$
S_{q1} (Horiz.)	9.00	8.3494	0.422	0.2818
S_{q2} (Vert.)	2.50	2.7624	0.7671	0.4549
S_{q3} (Diag.)	2.00	2.0471	0.3843	0.1754

The prompt given for the default options of fminbnd is shown below. More options were named but they were empty.

```
>> optimset('fminbnd')

ans =

struct with field:


    Display: 'notify'
    MaxFunEvals: 500
    MaxIter: 500
    TolFun: []
    TolX: 1.00000000000e-04
    FunValCheck: 'off'
```

The investigation into search direction S_{q2} revealed that reducing the termination tolerance `TolX` to $1.0e-8$ resulted in a 57% increase in function evaluations, rising from 26 to 41. While the default settings provided a sufficient approximation, the tighter tolerance refined the optimal step size α and the corresponding wire diameter d. As the algorithm converged, it increasingly used parabolic steps instead of golden. This procedure is generally faster than the golden section search as it fits a parabola to the points to estimate the minimum, but it requires the function to be sufficiently smooth.

Exercise 4

4.1 Steepest descent line searches using the algorithm fminbnd

In *springw4ex1.m* the point $\vec{x}_q = [D_0,\ d_0] = [0.022,\ 0.0035]$ is used as initial point. After running springw4ex1.m and carrying out some steepest descent line searches, it is clear that the results cannot be trusted. The optimizer is overshooting the target because of a scaling and step-size issue. The gradient *df* is calculated based on raw values, and since a small change in d (0.0035) affects the objective function much more than a small change in D (0.022), the search direction (sq) becomes dominated by one variable. Because the line search can look for a step size α between 0 and 1 without scaling the search vector sq, a step of 1 is massive.

To fix this, the search vector is scaled. A new file *springw4an1.m* is made where the search vector is scaled to unity length, then it is scaled to a reasonable length and a proper upper bound is used:

```
sq = -df;
% Normalize to unity length
sq = sq / norm(sq);
% Scale to reasonable length
scale = 0.01;
sq = sq * scale;
% Use a proper bound
alphamax = 10.0;
```

After running the program for several steps, the following results is shown:

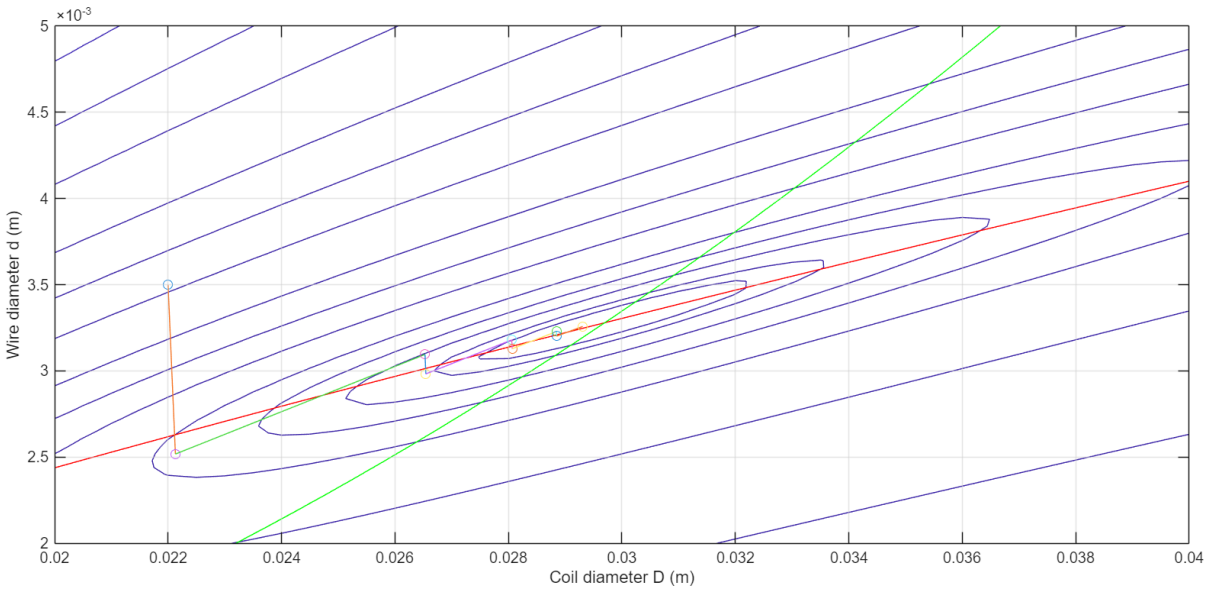

Figure 2: Line Search with Scaled Search Vector

The subsequent steepest descent search directions in fig. 2 are not orthogonal, but rather zig-zag. This is because one axis is stretched compared to the other in the visualization, and hence an actual 90° angle in the math becomes skewed in the figure.

4.2 Build your own steepest descent optimization algorithm

The program is extended to *springw4ex2* in order to automate the optimization process. The iteration is converged when the absolute difference between the current objective function value and the previous value is smaller than $1.0e-6$, or when more than 100 cycles are performed. The results are shown in fig. 3.

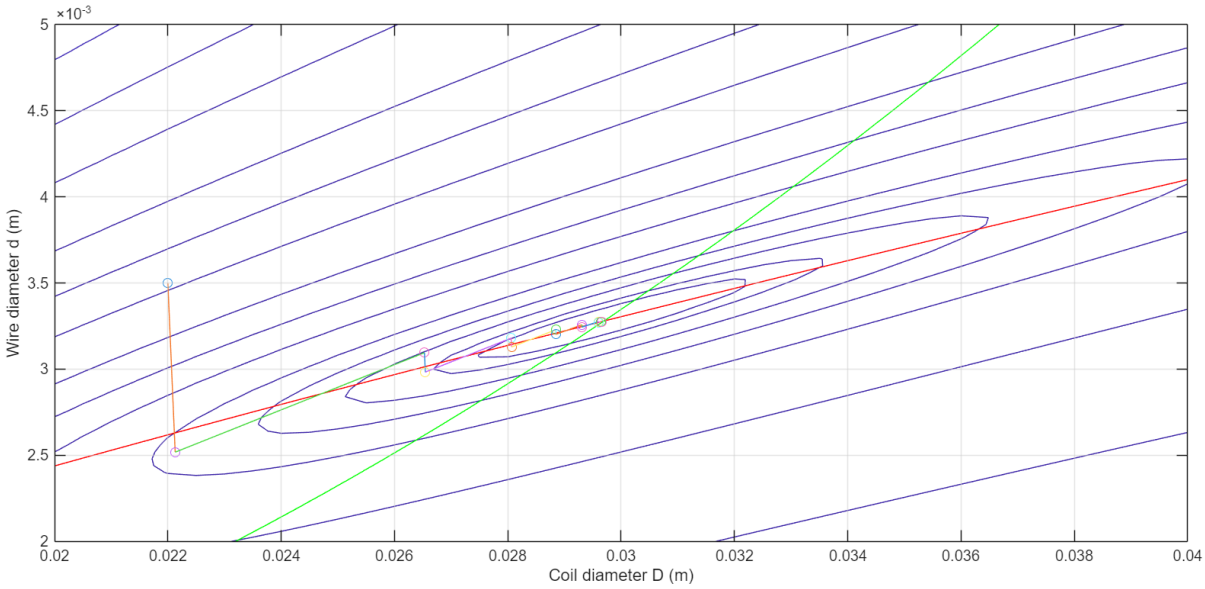

Figure 3: Automatic Optimization Process Results

The number of cycles under these conditions ($\alpha_{\max} = 10$ and scale $= 0.01$), the number of cycles required to reach convergence is 14. The final value of the objective function is $3.284741e-6$.

4.3 Solution the unconstrained problem using the algorithm fminunc

A new script *springw4ex3.m* is written using the Matlab algorithm *fminunc* to solve the problem of the previous subsections. The objective function has to be extracted from *sobjw43.m* because *fminunc* requires a function that accepts the full design vector $\vec{x}$ directly as its first input, which was not provided by *springobjw4.m*. The options used are as follows:

```
options = optimoptions('fminunc', ...
    'Algorithm', 'quasi-newton', ...
    'HessUpdate', 'BFGS', ...
    'Display', 'iter', ...
    'TolFun', 1e-6);
```

The results are shown in fig. 4. The efficiency was larger as this algorithm reached the optimal solution in just 10 iterations before it terminated first-order optimality fell below the tolerance. The final objective value is $1.059896e{-}9$, three orders of magnitude more precise than the previous results.

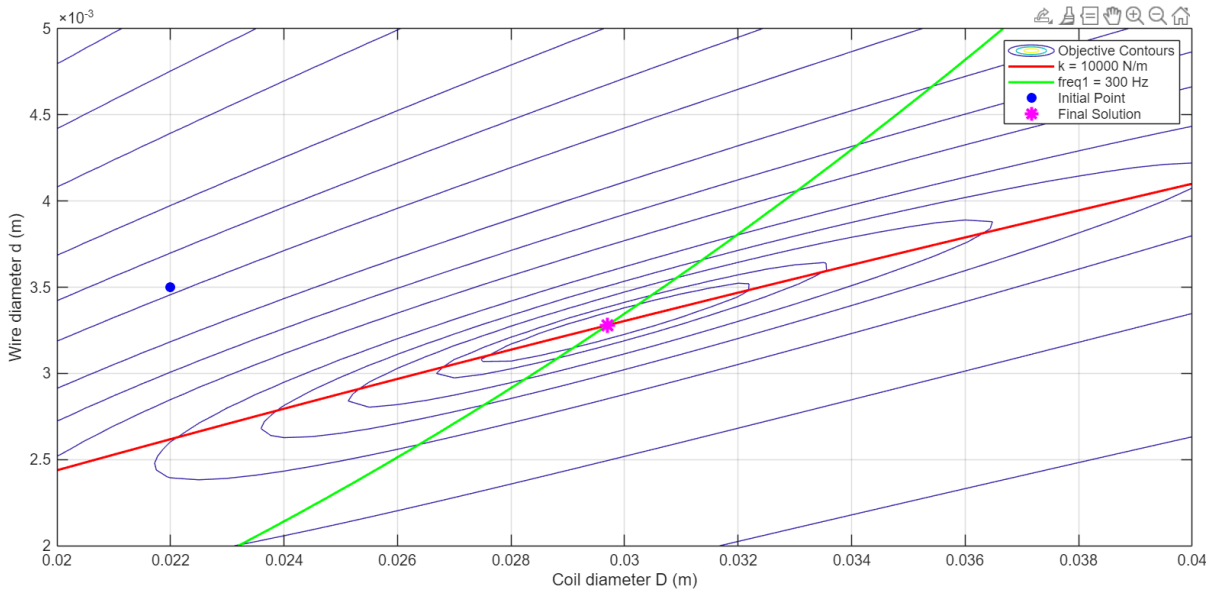

Figure 4: fminunc optimization results, BFGS Hessian Update

The algorithm is changed from BFGS to steepest descent. The results are shown in fig. 5. The change to this algorithm has increased the computational effort drastically, requiring 454 iterations to reach the optimal objective value of $9.26e{-}6$. In comparison to the self-made algorithm from section 4.2, the number of iterations has massively increase by over 30 times, and the final objective function is actually smaller (although the same order of magnitude). The difference is most likely due to the self-made algorithm using a large scaling factor for the search vector that forced the algorithm to take large steps towards the optimum, reaching the final optimal value quicker.

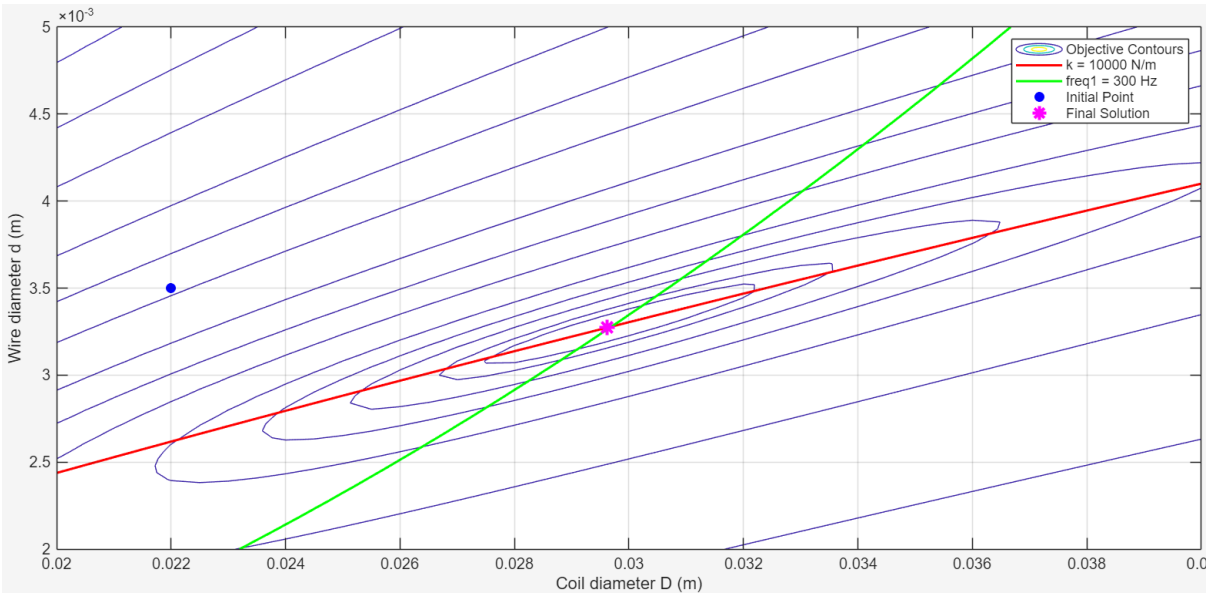

Figure 5: fminunc optimization results, steepest descent Hessian Update